

DevOps Project Summary

1. Project Overview

This project is a comprehensive DevOps infrastructure stack deployed using Docker Compose. It simulates a production-grade environment with load balancing, high-availability database replication, centralized logging, and infrastructure monitoring. The application is a simple Java-based visitor counter web app deployed on Apache Tomcat.

2. Architecture Components

2.1 Load Balancer — HAProxy

HAProxy (v2.8) sits at the front, listening on port 80 and distributing traffic across two Tomcat backends using round-robin balancing. Health checks are performed every 2 seconds. It also exposes Prometheus metrics on port 8404 and has a stats dashboard at `/stats`.

2.2 Application Servers — Apache Tomcat

Two Tomcat 9 (JDK 11) containers serve a Java visitor-counter web application. The app connects to PostgreSQL via Pgpool-II for database access. Both instances are identical and deployed in an active-active configuration.

2.3 Database Layer — PostgreSQL + Pgpool-II

PostgreSQL 15 (Alpine) runs in a master-slave streaming replication setup. Pgpool-II (v4.6.4) provides connection pooling, load balancing (read/write splitting), and failover between the master and slave nodes.

2.4 Centralized Logging — ELK Stack + Rsyslog

All services send logs via syslog to a central rsyslog container. Logstash reads these logs, parses them with Grok filters, and indexes them in Elasticsearch. Kibana provides a web UI for log visualization and search.

2.5 Monitoring — Prometheus + Grafana

Prometheus scrapes metrics from HAProxy, Tomcat (via a custom MetricsServlet), and a Node Exporter (for host-level metrics). Grafana provides dashboards to visualize all collected metrics.

3. Networking

All services run on a dedicated bridge network (devops-project_app-network). Port mappings:

Service	Host Port	Container Port
HAProxy	8082	80
HAProxy Metrics	8404	8404
Tomcat 1	8080	8080
Tomcat 2	8081	8080
Pgpool-II	5432	5432
Rsyslog	514 (UDP/TCP)	514
Elasticsearch	9200, 9300	9200, 9300
Kibana	5601	5601
Prometheus	9090	9090
Grafana	3000	3000

4. Tech Stack

- **Orchestration:** Docker Compose v3.8
- **Load Balancer:** HAProxy 2.8
- **App Server:** Apache Tomcat 9 + JDK 11
- **Database:** PostgreSQL 15 (Alpine) with streaming replication
- **Connection Pooling:** Pgpool-II 4.6.4
- **Log Aggregation:** Rsyslog → Logstash → Elasticsearch → Kibana
- **Monitoring:** Prometheus 2.45 + Grafana 10.0 + Node Exporter
- **Language:** Java 11 (Servlets + JSP)
- **Build Tool:** Maven

5. Directory Structure

Devops-project/

```
├─ docker-compose.yml      # Main orchestration file

├─ prometheus.yml          # Prometheus scrape config

├─ logstash.conf           # Logstash pipeline config

├─ haproxy/

|   └─ haproxy.cfg         # HAProxy config

├─ pgpool/

|   └─ Dockerfile          # Custom Pgpool-II image

|   └─ pgpool.conf         # Pgpool-II config

|   └─ pcp.conf            # PCP admin auth

|   └─ pool_passwd         # Pool password file

├─ rsyslog-config/

|   └─ rsyslog.conf        # Rsyslog config

├─ test-app/

|   └─ pom.xml             # Maven build file

|   └─ index.jsp           # JSP frontend

|   └─ src/main/java/com/
```

```
|      |— HelloServlet.java      # Main visitor-counter servlet

|      |— DBUtil.java           # Database connection utility

|      |— MetricsServlet.java   # Prometheus metrics endpoint

|      |— example/AppMetrics.java # Metrics tracking

|      |— logs/

|      |— remote.log            # Aggregated log output
```

6. Key Features

- High availability with HAProxy load balancing and health checks
- Database master-slave replication for fault tolerance
- Connection pooling and read/write splitting via Pgpool-II
- Centralized log aggregation with the full ELK stack
- Infrastructure monitoring with Prometheus and Grafana dashboards
- Custom Java servlet exposing Prometheus metrics for the application
- All services containerized and orchestrated with Docker Compose
- Scalable architecture — additional Tomcat replicas can be added easily

7. How to Run

1. Build the Java application: `cd test-app && mvn clean package`
2. Start all services: `docker compose up -d`
3. Access the app: <http://localhost:8082>
4. View logs: <http://localhost:5601> (Kibana)
5. View metrics: <http://localhost:9090> (Prometheus)
6. View dashboards: <http://localhost:3000> (Grafana, admin/admin)